

Module M203 - Developpement Front-End

Session 12 : Creation de Hooks Personnalisés

Objectifs de la Session

A la fin de cette session, vous serez capable de :

- Comprendre pourquoi et quand créer des hooks personnalisés
- Appliquer les règles des hooks React
- Créer des hooks réutilisables pour différents cas d'usage
- Implémenter les hooks courants : `useToggle`, `useLocalStorage`, `useFetch`, `useTheme`

1 Pourquoi Créer des Hooks Personnalisés ?

1.1 Le Probleme : Code Duplique

L'une des forces de React réside dans la possibilité de créer des **composants réutilisables**. Cependant, parfois la logique (et non l'interface) est dupliquée entre plusieurs composants.

Exemple de duplication :

```
1  // Composant A
2  function ComponentA() {
3    const [data, setData] = useState(null)
4    const [loading, setLoading] = useState(true)
5    const [error, setError] = useState(null)
6
7    useEffect(() => {
8      fetch('/api/users')
9        .then(res => res.json())
10       .then(data => { setData(data); setLoading(false) })
11       .catch(err => { setError(err); setLoading(false) })
12    }, [])
13    // ...
14  }
15
16  // Composant B - MEME LOGIQUE !
17  function ComponentB() {
18    const [data, setData] = useState(null)
19    const [loading, setLoading] = useState(true)
20    const [error, setError] = useState(null)
21
22    useEffect(() => {
23      fetch('/api/products') // Seule l'URL change
24        .then(res => res.json())
25        .then(data => { setData(data); setLoading(false) })
26        .catch(err => { setError(err); setLoading(false) })
```

```
27   }, [])  
28   // ...  
29 }
```

1.2 La Solution : Custom Hooks

Les **Hooks personnalisés** permettent d'extraire et de réutiliser la logique entre composants.

Definition

Un **Custom Hook** est une fonction JavaScript dont le nom commence par **use** et qui peut appeler d'autres hooks React.

Avantages :

- **DRY** (Don't Repeat Yourself) : Eviter la duplication de code
- **Lisibilité** : Code plus propre et compréhensible
- **Testabilité** : Logique isolée, facile à tester
- **Maintenance** : Un seul endroit à modifier

2 Regles des Hooks

Regles Obligatoires

1. **Prefixe "use"** : Le nom doit commencer par **use** (ex: `useFetch`, `useTheme`)
2. **Top Level Only** : Appeler les hooks uniquement au niveau supérieur (pas dans des conditions, boucles, ou fonctions imbriquées)
3. **Composants React uniquement** : Appeler les hooks uniquement dans des composants fonctionnels React ou d'autres hooks personnalisés

```
1  // CORRECT  
2  function useMyHook() {  
3    const [state, setState] = useState(0)    // Top level  
4    useEffect(() => { ... }, [])            // Top level  
5    return state  
6  }  
7  
8  // INCORRECT  
9  function useMyHook(condition) {  
10   if (condition) {  
11     const [state, setState] = useState(0)    // Dans une condition  
12   }  
13 }
```

3 Hook useToggle (Simple)

Le hook le plus simple : gerer un etat booleen avec une fonction pour le basculer.

3.1 Implementation

```
1 // hooks/useToggle.js
2 import { useState } from 'react'
3
4 function useToggle(initialValue = false) {
5   const [value, setValue] = useState(initialValue)
6
7   const toggle = () => {
8     setValue(prevValue => !prevValue)
9   }
10
11   return [value, toggle]
12 }
13
14 export default useToggle
```

3.2 Utilisation

```
1 import useToggle from './hooks/useToggle'
2
3 function Modal() {
4   const [isOpen, toggleOpen] = useToggle(false)
5
6   return (
7     <div>
8       <button onClick={toggleOpen}>
9         {isOpen ? 'Fermer' : 'Ouvrir'} Modal
10       </button>
11       {isOpen && (
12         <div className="modal">
13           <p>Contenu du modal</p>
14           <button onClick={toggleOpen}>Fermer</button>
15         </div>
16       )}
17     </div>
18   )
19 }
```

4 Hook useLocalStorage (Persistance)

Persister des donnees dans le localStorage du navigateur.

4.1 Implementation

```
1 // hooks/useLocalStorage.js
2 import { useState, useEffect } from 'react'
3
4 function useLocalStorage(key, initialValue) {
5   // Initialiser avec la valeur du localStorage ou la valeur par
6   // défaut
7   const [storedValue, setStoredValue] = useState(() => {
8     try {
9       const item = localStorage.getItem(key)
10      return item ? JSON.parse(item) : initialValue
11    } catch (error) {
12      console.error(error)
13      return initialValue
14    }
15  })
16
17   // Mettre a jour le localStorage quand la valeur change
18   useEffect(() => {
19     try {
20       localStorage.setItem(key, JSON.stringify(storedValue))
21     } catch (error) {
22       console.error(error)
23     }
24   }, [key, storedValue])
25
26   return [storedValue, setStoredValue]
27 }
28 export default useLocalStorage
```

4.2 Utilisation

```
1 import useLocalStorage from '../hooks/useLocalStorage'
2
3 function Settings() {
4   const [username, setUsername] = useLocalStorage('username', '')
5   const [darkMode, setDarkMode] = useLocalStorage('darkMode', false)
6
7   return (
8     <div>
9       <input
10         value={username}
11         onChange={(e) => setUsername(e.target.value)}
12         placeholder="Nom d'utilisateur"
13       />
14       <p>Bonjour, {username || 'Visiteur'}</p>
15     </div>
16   )
17 }
```

```
15
16     <button onClick={() => setDarkMode(!darkMode)}>
17         Mode: {darkMode ? 'Sombre' : 'Clair'}
18     </button>
19 </div>
20 )
21 }
```

Note

Les données persistent même après fermeture du navigateur. Idéal pour les préférences utilisateur.

5 Hook useFetch (Requetes HTTP)

Le hook le plus utile : centraliser la logique de fetch avec gestion loading/error.

5.1 Implementation Basique (GET)

```
1  // hooks/useFetch.js
2  import { useState, useEffect } from 'react'
3
4  function useFetch(url) {
5      const [data, setData] = useState(null)
6      const [loading, setLoading] = useState(true)
7      const [error, setError] = useState(null)
8
9      useEffect(() => {
10         const fetchData = async () => {
11             setLoading(true)
12             setError(null)
13
14             try {
15                 const response = await fetch(url)
16
17                 if (!response.ok) {
18                     throw new Error('Erreur HTTP: ${response.status}')
19                 }
20
21                 const result = await response.json()
22                 setData(result)
23             } catch (err) {
24                 setError(err.message)
25             } finally {
26                 setLoading(false)
27             }
28         }
29
30         fetchData()
```

```
31   }, [url])
32
33   return { data, loading, error }
34 }
35
36 export default useFetch
```

5.2 Utilisation

```
1  import useFetch from './hooks/useFetch'
2
3  function UsersList() {
4    const { data: users, loading, error } = useFetch(
5      'https://jsonplaceholder.typicode.com/users'
6    )
7
8    if (loading) return <p>Chargement...</p>
9    if (error) return <p style={{color: 'red'}}>Erreur: {error}</p>
10
11    return (
12      <ul>
13        {users.map(user => (
14          <li key={user.id}>{user.name} - {user.email}</li>
15        ))}
16      </ul>
17    )
18  }
```

5.3 Implementation Avancee (Toutes Methodes HTTP)

```
1  // hooks/useHttp.js
2  import { useState, useCallback } from 'react'
3
4  function useHttp() {
5    const [loading, setLoading] = useState(false)
6    const [error, setError] = useState(null)
7
8    const sendRequest = useCallback(async (url, method = 'GET', body
9      = null) => {
10      setLoading(true)
11      setError(null)
12
13      try {
14        const options = {
15          method,
16          headers: {
17            'Content-Type': 'application/json'
18          }
19        }
```

```
18     }
19
20     if (body) {
21       options.body = JSON.stringify(body)
22     }
23
24     const response = await fetch(url, options)
25
26     if (!response.ok) {
27       throw new Error(`Erreur HTTP: ${response.status}`)
28     }
29
30     const data = await response.json()
31     setLoading(false)
32     return data
33   } catch (err) {
34     setError(err.message)
35     setLoading(false)
36     throw err
37   }
38 }, [])
39
40 return { loading, error, sendRequest }
41 }
42
43 export default useHttp
```

5.4 Utilisation avec POST, PUT, DELETE

```
1 import useHttp from './hooks/useHttp'
2
3 function ProductManager() {
4   const { loading, error, sendRequest } = useHttp()
5   const [products, setProducts] = useState([])
6
7   // GET - Charger les produits
8   const fetchProducts = async () => {
9     const data = await sendRequest('/api/products')
10    setProducts(data)
11  }
12
13  // POST - Ajouter un produit
14  const addProduct = async (newProduct) => {
15    const created = await sendRequest('/api/products', 'POST',
16      newProduct)
17    setProducts([...products, created])
18  }
19
20  // PUT - Modifier un produit
```

```

20  const updateProduct = async (id, updatedData) => {
21    const updated = await sendRequest(`/api/products/${id}`, 'PUT',
      updatedData)
22    setProducts(products.map(p => p.id === id ? updated : p))
23  }
24
25  // DELETE - Supprimer un produit
26  const deleteProduct = async (id) => {
27    await sendRequest(`/api/products/${id}`, 'DELETE')
28    setProducts(products.filter(p => p.id !== id))
29  }
30
31  return (
32    <div>
33      {loading && <p>Chargement...</p>}
34      {error && <p style={{color: 'red'}}>{error}</p>}
35      {/* ... UI ... */}
36    </div>
37  )
38  }

```

6 Hook useTheme (Gestion du Theme)

Gerer le theme clair/sombre de l'application.

6.1 Implementation

```

1  // hooks/useTheme.js
2  import { useState, useEffect } from 'react'
3
4  function useTheme(defaultTheme = 'light') {
5    // Initialiser avec le theme stocke ou le default
6    const [theme, setTheme] = useState(() => {
7      const saved = localStorage.getItem('theme')
8      return saved || defaultTheme
9    })
10
11    // Fonction pour basculer le theme
12    const toggleTheme = () => {
13      setTheme(prevTheme => prevTheme === 'light' ? 'dark' : 'light')
14    }
15
16    // Mettre a jour le localStorage et le body class
17    useEffect(() => {
18      localStorage.setItem('theme', theme)
19      document.body.className = `theme-${theme}`
20    }, [theme])
21
22    return { theme, toggleTheme, setTheme }

```



```
23 }  
24  
25 export default useTheme
```

6.2 Utilisation

```
1 // App.jsx  
2 import useTheme from '../hooks/useTheme'  
3  
4 function App() {  
5   const { theme, toggleTheme } = useTheme('light')  
6  
7   return (  
8     <div className={`app theme-${theme}`}>  
9       <header>  
10        <h1>Mon Application</h1>  
11        <button onClick={toggleTheme}>  
12          {theme === 'light' ? 'Mode Sombre' : 'Mode Clair'}  
13        </button>  
14      </header>  
15      <main>  
16        <p>Theme actuel : {theme}</p>  
17      </main>  
18    </div>  
19  )  
20 }
```

6.3 CSS pour les Themes

```
1 /* styles.css */  
2 .theme-light {  
3   --bg-color: #ffffff;  
4   --text-color: #333333;  
5   --primary-color: #007bff;  
6 }  
7  
8 .theme-dark {  
9   --bg-color: #1a1a2e;  
10  --text-color: #ffffff;  
11  --primary-color: #4dabf7;  
12 }  
13  
14 body {  
15   background-color: var(--bg-color);  
16   color: var(--text-color);  
17   transition: all 0.3s ease;  
18 }
```

7 Organisation des Hooks

7.1 Structure Recommandee

```
src/  
  hooks/  
    useToggle.js  
    useLocalStorage.js  
    useFetch.js  
    useHttp.js  
    useTheme.js  
    index.js      <- Barrel export  
  components/  
  pages/  
  App.jsx
```

7.2 Barrel Export

```
1 // hooks/index.js  
2 export { default as useToggle } from './useToggle'  
3 export { default as useLocalStorage } from './useLocalStorage'  
4 export { default as useFetch } from './useFetch'  
5 export { default as useHttp } from './useHttp'  
6 export { default as useTheme } from './useTheme'  
7  
8 // Utilisation  
9 import { useFetch, useTheme } from './hooks'
```

8 Resume - Syntaxe a Retenir

Syntaxe pour Examen

1. Structure d'un Custom Hook :

```
1 import { useState, useEffect } from 'react'
2
3 function useMyHook(param) {
4   const [state, setState] = useState(initialValue)
5
6   useEffect(() => {
7     // Side effect
8   }, [param])
9
10  return { state, setState } // ou [state, setState]
11 }
12
13 export default useMyHook
```

2. useFetch minimal :

```
1 function useFetch(url) {
2   const [data, setData] = useState(null)
3   const [loading, setLoading] = useState(true)
4   const [error, setError] = useState(null)
5
6   useEffect(() => {
7     fetch(url)
8       .then(res => res.json())
9       .then(setData)
10      .catch(err => setError(err.message))
11      .finally(() => setLoading(false))
12   }, [url])
13
14   return { data, loading, error }
15 }
```

3. useToggle minimal :

```
1 function useToggle(initial = false) {
2   const [value, setValue] = useState(initial)
3   const toggle = () => setValue(prev => !prev)
4   return [value, toggle]
5 }
```

Ressources

- React Documentation - Custom Hooks : <https://react.dev/learn/reusing-logic-with-c>

- **useHooks** (Collection de hooks) : <https://usehooks.com/>
- **W3Schools React Hooks** : https://www.w3schools.com/react/react_customhooks.asp
- **GeeksForGeeks Custom Hooks** : <https://www.geeksforgeeks.org/reactjs-custom-hooks/>